

GUIDE EXHAUSTIF DE L'ARCHITECTURE, DE L'OPTIMISATION ET DE L'ENTRAÎNEMENT EN APPRENTISSAGE PROFOND

Analyse rigoureuse des Logits, de la fonction Softmax, du rôle distinct entre la Loss Function et l'Optimizer, du Batching, de la Rétropropagation et du catalogue complet des Hyperparamètres.

Domaine : Intelligence Artificielle / Deep Learning

Document : Manuel Technique & Theorique

Niveau : Avancé / Master & Recherche

1. Introduction Généraliste & Contexte de l'Apprentissage Supervisé

L'apprentissage profond (*Deep Learning*) a révolutionné le domaine de l'intelligence artificielle en permettant aux réseaux de neurones d'apprendre des représentations hiérarchiques complexes directement à partir de données brutes. Qu'il s'agisse de classification d'images, de traitement du langage naturel (NLP) ou de prédiction de séries temporelles, le cœur du processus d'apprentissage repose sur une boucle d'optimisation mathématique continue.

Pour comprendre comment une machine acquiert une compétence, il est essentiel d'aborder la frontière précise où le réseau passe de la simple extraction de caractéristiques (représentations internes) à la prise de décision probabiliste, puis à la correction de ses propres erreurs. Ce document propose un décortilage exhaustif et mathématiquement rigoureux de toutes les étapes composant ce cycle, depuis les sorties brutes du réseau jusqu'à la mise à jour fine de ses paramètres.

2. Sortie du Réseau : Des Logits à la Distribution de Probabilité (Softmax)

2.1. Nature et Définition des Logits

À la toute dernière couche d'un réseau de neurones artificiels dédié à la classification, chaque neurone de sortie effectue une combinaison affine de ses entrées, suivie ou non d'une pré-activation. Pour un vecteur d'entrée de la dernière couche $\mathbf{h}$, de matrice de poids $\mathbf{W}$ et de biais $\mathbf{b}$, la sortie brute s'exprime sous la forme :

$$z_i = \mathbf{W}_i^T \mathbf{h} + b_i$$

Ces valeurs z_i sont appelées les **Logits** (ou scores bruts non normalisés). La propriété fondamentale des Logits est qu'ils appartiennent à l'ensemble des nombres réels : $z_i \in (-\infty; +\infty)$.

Considérons un problème de classification à 3 classes (Chien, Chat, Oiseau). Le réseau produit un vecteur de logits :

$$z = [z_1, z_2, z_3]^T = [4.5, -1.2, 0.8]^T$$

Ces nombres sauvages posent deux problèmes majeurs pour un système décisionnel :

- **Absence de bornes** : Un score de 4.5 ne possède aucune signification intrinsèque absolue. Il dépend de l'échelle des poids W . On ne peut pas affirmer directement qu'un score de 4.5 correspond à une certitude de 90% ou 99%.
- **Absence de normalisation mutuelle** : La somme des logits n'est pas contrainte à évaluer 1 ($4.5 - 1.2 + 0.8 = 4.1 \neq 1.0$). Il est donc impossible de les interpréter directement comme des probabilités au sens des axiomes de Kolmogorov.

2.2. La Fonction Softmax : Définition Mathématique et Rôle

Pour transformer ce vecteur de scores réels $z \in \mathbb{R}^K$ en un vecteur de probabilités $\hat{y} \in [0, 1]^K$ tel que $\sum_{i=1}^K \hat{y}_i = 1$, on utilise la fonction d'activation **Softmax** (également appelée fonction d'activation exponentielle normalisée).

La formule mathématique pour la i -ème composante du vecteur de sortie est donnée par :

$$\hat{y}_i = \text{Softmax}(z)_i = e^{z_i} / \sum_{j=1}^K e^{z_j}$$

La fonction Softmax opère en deux temps distincts :

1. **Passage à l'exponentielle (e^{z_i})** : L'application de la fonction exponentielle garantit que toutes les valeurs deviennent strictement positives ($e^{z_i} > 0$), éliminant ainsi les nombres négatifs. De plus, l'exponentielle accentue fortement les écarts entre le score le plus élevé et les autres scores (effet "max").
2. **Normalisation ($\sum e^{z_j}$)** : La division par la somme des exponentielles de l'ensemble des logits ramène la somme totale de toutes les composantes à exactement 1 (ou 100%).

2.3. Exemple Numérique Détaillé

Reprenons notre vecteur de logits $z = [4.5, -1.2, 0.8]$:

- Calcul des exponentielles :
 - $e^{4.5} \approx 90.017$
 - $e^{-1.2} \approx 0.301$
 - $e^{0.8} \approx 2.226$

- Somme des exponentielles :

$$S = 90.017 + 0.301 + 2.226 = 92.544$$

- Calcul des probabilités individuelles $\hat{y}_i$:
 - $\hat{y}_1$ (Chien) = $90.017 / 92.544 \approx 0.9727$ (97.27%)
 - $\hat{y}_2$ (Chat) = $0.301 / 92.544 \approx 0.0033$ (0.33%)
 - $\hat{y}_3$ (Oiseau) = $2.226 / 92.544 \approx 0.0240$ (2.40%)

La somme $0.9727 + 0.0033 + 0.0240 = 1.0000$ est rigoureusement respectée. Le réseau fournit une distribution de probabilité exploitable.

Stabilité Numérique de la Softmax (Softmax Trick)

En pratique informatique (float32/float64), si un logit z_i est très grand (ex: $z_i = 1000$), e^{1000} provoque un dépassement de capacité (*overflow*). Inversement, des valeurs très négatives risquent d'entraîner un *underflow*. Pour éviter cela, les bibliothèques d'IA soustraient le maximum du vecteur $M = \max_j(z_j)$ avant le calcul :

$$\text{Softmax}(z)_i = e^{(z_i - M)} / \sum_j e^{(z_j - M)}$$

Cette transformation est rigoureusement identique sur le plan mathématique car le facteur e^{-M} s'annule au numérateur et au dénominateur, mais elle garantit une parfaite stabilité numérique.

3. Séparation Clarifiée des Rôles : Loss Function vs. Optimizer

L'une des confusions les plus fréquentes chez les débutants en Deep Learning réside dans la frontière entre la fonction de perte (*Loss Function*) et l'optimiseur (*Optimizer*). Ces deux composants interviennent dans la phase de correction du réseau mais possèdent des responsabilités totalement distinctes.

3.1. La Fonction de Perte (Loss Function) : Le Mesureur d'Erreur (Le Juge)

La **Loss Function** (ou fonction de coût $J(\theta)$ sur l'ensemble du jeu de données) est une fonction mathématique scalaire. Son rôle **unique et exclusif** est d'évaluer la qualité de la prédiction du réseau en calculant l'écart entre la prédiction $\hat{y}$ et la cible réelle (la vérité terrain) y .

- **Entrées** : La prédiction $\hat{y}$ (ex: probabilités Softmax) et la vraie étiquette y (ex: vecteur *one-hot*).
- **Sortie** : Un unique nombre réel positif $L \geq 0$. Plus ce nombre est proche de zéro, plus le réseau est performant.
- **Ce qu'elle NE FAIT PAS** : La Loss Function **ne modifie aucun poids**. Elle ne connaît pas la structure des couches, n'a pas accès à la mémoire des gradients et n'effectue aucune mise à jour des paramètres. Elle agit strictement comme un thermomètre ou un juge impartial.

3.2. L'Optimiseur (Optimizer) : Le Mécanicien du Réseau

L'**Optimiseur** est l'algorithme de recherche globale qui prend les décisions de modification de la mémoire du réseau. Il utilise l'information fournie par la Loss Function (via la dérivée/gradient) pour altérer la valeur des poids **W** et des biais **b**.

- **Rôle** : Déterminer *comment* et *de combien* chaque poids doit varier pour minimiser la Loss lors des prochains passages.
- **Mécanisme** : L'optimiseur met en œuvre l'algorithme de **Rétropropagation du Gradient** (*Backpropagation*) pour calculer le gradient $\nabla_{\mathbf{W}} L$, puis applique sa propre règle de mise à jour (ex: SGD, Adam, RMSprop) en ajustant les pas selon son taux d'apprentissage et ses moments historiques.

Caractéristique	Loss Function (Fonction de Perte)	Optimizer (Optimiseur)
Analogie	Le Juge / Le Thermomètre	Le Coach / Le Mécanicien
Rôle Principal	Mesurer la gravité de l'erreur commise	Ajuster les poids W pour réduire l'erreur
Nature Mathématique	Fonction d'évaluation scalaire $L(\hat{y}, y)$	Algorithme de mise à jour paramétrique $\mathbf{W} \leftarrow \mathbf{W} - \Delta \mathbf{W}$
Exemples Classiques	Categorical Cross-Entropy, MSE, BCE	SGD, Adam, RMSprop, Adagrad
Action sur les Poids	Aucune action directe	Modification directe des matrices de poids

4. Formulation Mathématique Complète de la Boucle d'Entraînement

Pour comprendre l'enchaînement exact des opérations, suivons rigoureusement les trois étapes mathématiques qui constituent une itération d'entraînement pour un problème de classification multi-classes.

Étape A : Transformation Probabiliste (Softmax)

Soit z_i le logit produit par le neurone **i** pour une instance donnée. Le vecteur de prédiction probabiliste est :

$$\hat{y}_i = e^{z_i} / \sum_{j=1}^K e^{z_j}$$

Étape B : Mesure de l'Erreur (Categorical Cross-Entropy)

La fonction de perte standard pour la classification multi-classes est l'**Entropie Croisée Catégorielle** (*Categorical Cross-Entropy*). Soit y le vecteur réel encodé en *one-hot* (ex: $y = [1, 0, 0]^T$ pour un chien) et $\hat{y}$ le vecteur de probabilités prédites. La perte pour un exemple L est :

$$L(\hat{y}, y) = - \sum_{i=1}^K y_i \cdot \log(\hat{y}_i)$$

Puisque y est un vecteur *one-hot* où seule la classe cible c vaut 1 (et toutes les autres zéro), la formule se simplifie en :

$$L(\hat{y}, y) = - \log(\hat{y}_c)$$

Analyse de comportement :

- Si $\hat{y}_c \rightarrow 1$ (prédiction très confiante et correcte), $\log(1) = 0$, donc $L \rightarrow 0$ (erreur quasi nulle).
- Si $\hat{y}_c \rightarrow 0$ (prédiction incorrecte), $\log(0) \rightarrow -\infty$, donc $L \rightarrow +\infty$ (pénalité géante).

Étape C : Rétropropagation et Mise à Jour des Poids (Descente de Gradient)

Pour mettre à jour un poids quelconque w du réseau, l'optimiseur doit connaître le gradient de la perte par rapport à ce poids : $\partial L / \partial w$. En vertu de la règle de dérivation en chaîne (*Chain Rule*) :

$$\partial L / \partial w = (\partial L / \partial \hat{y}) \cdot (\partial \hat{y} / \partial z) \cdot (\partial z / \partial w)$$

Fait remarquable : lorsque l'on combine la dérivée de la Softmax avec celle de l'entropie croisée, le gradient direct du logit z_i se simplifie de façon extrêmement élégante :

$$\partial L / \partial z_i = \hat{y}_i - y_i$$

Ce résultat intuitif montre que l'erreur transmise en arrière est exactement la différence simple entre la probabilité prédite et la vérité terrain !

L'algorithme de l'optimiseur (ici la Descente de Gradient Stochastique) applique ensuite la formule de mise à jour fondamentale :

$$w^{(\text{nouveau})} = w^{(\text{ancien})} - \alpha \cdot (\partial L / \partial w)$$

où α représente le taux d'apprentissage (*Learning Rate*).

5. Visualisation Schématique Globale de l'Apprentissage

Le schéma ci-dessous illustre l'architecture complète du flux d'apprentissage : du passage avant (*Forward Pass*) à travers les couches cachées jusqu'au calcul de la perte et à la rétropropagation du gradient (*Backward Pass*) orchestrée par l'optimiseur.

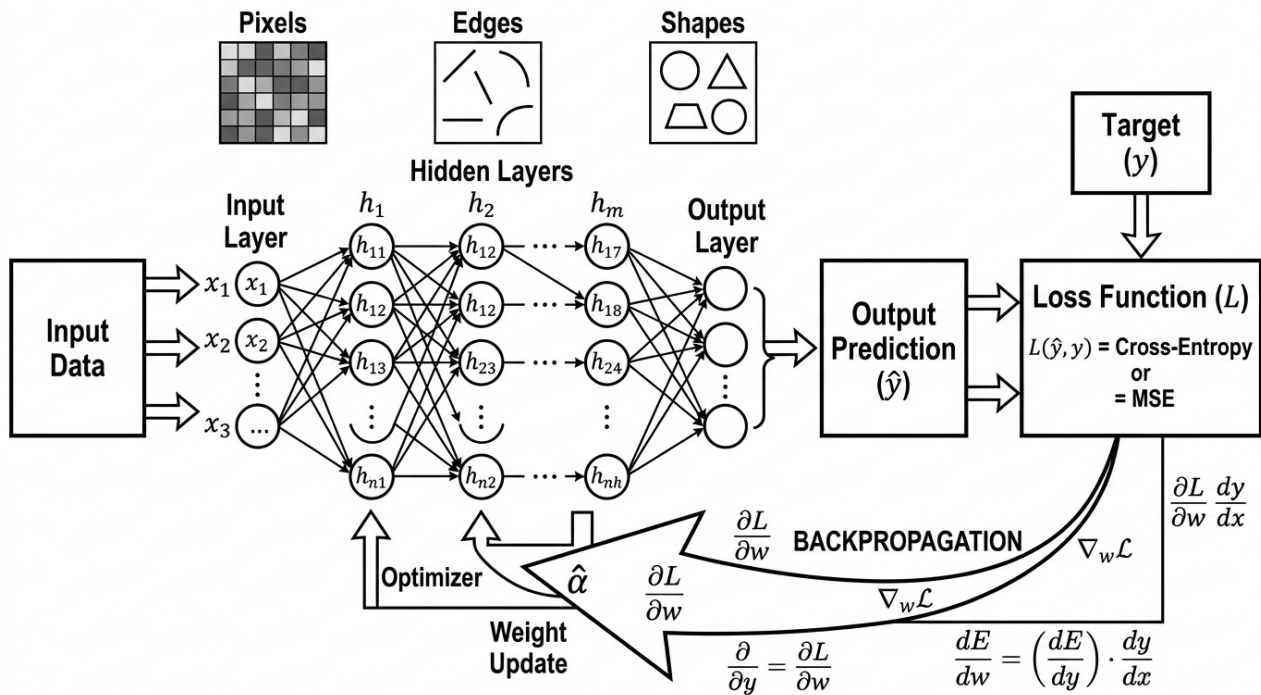


Figure 1 : Visualisation complète de la boucle d'entraînement d'un réseau de neurones. Le flux avant (*Forward*) extrait les caractéristiques et produit la prédiction $\hat{y}$. La *Loss Function* évalue la perte L par rapport à y . La rétropropagation (*Backward*) calcule les gradients $\partial L / \partial w$, et l'optimiseur applique la correction pondérée par α .

6. Fonctionnement Interne d'un Réseau : Batch Size, Couches et Neurones

Pour lever toute ambiguïté sur la question spécifique posée ("*batch_size = 32* veut-il dire que certains neurones travaillent et d'autres sont éteints ? Quand la 1ère couche finit, passe-t-on à la suivante ?"), analysons le comportement spatio-temporel du réseau.

6.1. Clarification Cruciale : Le Rôle du Batch Size (32 exemples)

Idée Reçue vs Réalité Physique

Non, un `batch_size` de 32 ne signifie ABSOLUMENT PAS que certains neurones travaillent et d'autres sont éteints ! Tous les neurones d'une couche travaillent simultanément pour chaque donnée.

Le **Batch Size** définit le nombre d'**échantillons/exemples de données** (ex: 32 images de chiens et chats) qui sont traités *en parallèle* par l'ordinateur (GPU/CPU) avant de procéder à une seule mise à jour des poids du réseau.

- Si vous avez un jeu de données de 10 000 images et un **batch_size** = 32, votre jeu de données est découpé en $10\,000 / 32 = 312.5$ paquets (lots).
- Pour chaque lot de 32 images, les 32 images traversent l'intégralité du réseau en même temps sous forme de calcul matriciel tensoriel.
- Le réseau calcule les prédictions pour ces 32 images, calcule l'**erreur moyenne** de ces 32 images, puis l'optimiseur met à jour les poids **une seule fois** pour ce lot.

6.2. La Propagation par Couches (Layer by Layer)

Quant au fonctionnement de couche en couche : **Oui, le traitement est séquentiel au sein d'un passage avant (Forward Pass).**

1. **Couche d'Entrée (Input Layer)** : Reçoit la matrice des pixels des 32 images simultanément (tenseur de forme $[32, C, H, W]$).
2. **Première Couche Cachée (h_1)** : Effectue le calcul matriciel $Z_1 = X W_1 + b_1$ pour l'ensemble du lot de 32 images, applique la fonction d'activation (ex: ReLU), et transmet le résultat A_1 à la couche suivante.
3. **Deuxième Couche Cachée (h_2)** : Attend la fin du calcul de h_1 , puis calcule $Z_2 = A_1 W_2 + b_2$. Et ainsi de suite jusqu'à la couche de sortie.

Les neurones "éteints" au hasard ne sont pas dus au `batch_size`, mais à une technique spécifique appelée **Dropout** (détaillée au Chapitre 7).

7. Catalogue Exhaustif des Hyperparamètres et Algorithmes en Deep Learning

Lors de la conception d'un pipeline d'apprentissage profond, l'ingénieur doit configurer un ensemble de paramètres de contrôle appelés **Hyperparamètres** (car ils ne sont pas appris par le réseau lui-même, mais fixés avant l'entraînement). Voici l'inventaire complet et systématique de tous les hyperparamètres et algorithmes existants.

7.1. Paramètres de Temporel et de Gestion des Données

- **Epochs (Époques)** : Nombre total de passages complets de l'intégralité du jeu de données d'entraînement à travers le réseau.
Exemple : 50 époques signifie que chaque image de la base aura été vue 50 fois par l'algorithme.
- **Batch Size (Taille du Lot)** : Nombre d'exemples de données traités simultanément en mémoire vectorielle avant la mise à jour des poids.
Valeurs courantes : 16, 32, 64, 128, 256, 512 (puissances de 2 privilégiées pour l'optimisation mémoire GPU).
- **Steps per Epoch (Pas par Époque)** : Nombre total de lots à traiter pour valider une époque complète. **Steps** = $N / \text{batch_size}$ (où N est le nombre total d'échantillons).

- **Shuffle (Mélange)** : Booléen indiquant si les données doivent être réordonnées aléatoirement au début de chaque époque pour éviter que le réseau n'apprenne l'ordre de présentation des données.

7.2. Algorithmes d'Optimisation (Les Optimiseurs)

L'optimiseur choisit la trajectoire dans l'espace des paramètres pour atteindre le minimum de la surface d'erreur.

1. Stochastic Gradient Descent (SGD)

La forme la plus basique. Met à jour les poids strictement dans la direction opposée du gradient instantané.

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} J(\theta_t)$$

Inconvénients : Très lent, sensible aux points cols (*saddle points*) et aux oscillations dans les vallées ravines.

2. SGD avec Momentum

Ajoute une fraction du vecteur d'actualisation de l'étape précédente (inertie), telle une bille roulant sur une pente qui accumule de la vitesse.

$$\begin{aligned} v_t &= \gamma v_{t-1} + \alpha \nabla_{\theta} J(\theta_t) \\ \theta_{t+1} &= \theta_t - v_t \end{aligned}$$

où γ est le coefficient de momentum (souvent 0.9).

3. Nesterov Accelerated Gradient (NAG)

Une variante intelligente du Momentum qui calcule le gradient non pas à la position actuelle, mais à la position future approximée ($\theta - \gamma v_{t-1}$), permettant d'anticiper les freinages.

4. RMSprop (Root Mean Squared Propagation)

Algorithme à taux d'apprentissage adaptatif. Il divise le taux d'apprentissage de chaque paramètre par la moyenne mobile de la racine carrée des gradients au carré récents.

$$\begin{aligned} v_t &= \beta v_{t-1} + (1 - \beta) (\nabla J(\theta_t))^2 \\ \theta_{t+1} &= \theta_t - (\alpha / (\sqrt{\beta v_t + \epsilon})) \cdot \nabla J(\theta_t) \end{aligned}$$

5. Adam (Adaptive Moment Estimation)

Le standard industriel actuel. Combine les avantages du Momentum (estimation du premier moment / moyenne) et de RMSprop (estimation du second moment / variance non centrée).

$$\begin{aligned}
m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t && \text{(Premier moment)} \\
v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 && \text{(Second moment)} \\
\hat{m}_t &= m_t / (1 - \beta_1^t), \quad \hat{v}_t = v_t / (1 - \beta_2^t) && \text{(Correction du biais)} \\
\theta_{t+1} &= \theta_t - (\alpha / (\sqrt{\hat{v}_t} + \epsilon)) \cdot \hat{m}_t
\end{aligned}$$

Hyperparamètres d'Adam : $\alpha = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$.

6. AdamW

Modifie l'algorithme Adam en séparant correctement la régularisation par découplage du déclin de poids (*Weight Decay*) du calcul du gradient adaptatif, offrant une meilleure généralisation sur les Transformers et CNNs modernes.

7.3. Fonctions de Perte (Loss Functions)

Nom de la Loss	Domaine d'Application	Formule Mathématique
Categorical Cross-Entropy	Classification multi-classes (labels <i>one-hot</i>)	$L = - \sum y_i \log(\hat{y}_i)$
Sparse Categorical Cross-Entropy	Classification multi-classes (labels sous forme d'entiers)	Identique, mais évite la conversion explicite en <i>one-hot</i>
Binary Cross-Entropy (BCE)	Classification binaire (0 ou 1, vrai/faux)	$L = - [y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$
Mean Squared Error (MSE)	Régression (prédiction de valeurs continues)	$L = (1/n) \sum (y_i - \hat{y}_i)^2$
Mean Absolute Error (MAE)	Régression robuste aux valeurs aberrantes (<i>outliers</i>)	$L = (1/n) \sum y_i - \hat{y}_i $
Huber Loss / Smooth L1	Régression hybride (MSE près de 0, MAE ailleurs)	Combinaison quadratique et linéaire
Focal Loss	Classification avec fort déséquilibre de classes	$L = - \alpha_i (1 - p_i)^\gamma \log(p_i)$

7.4. Fonctions d'Activation et Régularisation d'Architecture

1. ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

La fonction la plus populaire pour les couches cachées. Très simple à calculer, évite la saturation des gradients pour les valeurs positives. Risque : *Dying ReLU* (neurones définitivement éteints si $x < 0$).

2. Leaky ReLU & PReLU

$$f(x) = \max(\alpha x, x) \quad (\text{avec } \alpha \approx 0.01)$$

Permet un léger passage de gradient pour les valeurs négatives afin d'éviter le problème des neurones morts.

3. Sigmoid & Tanh

$$\sigma(x) = 1 / (1 + e^{-x}) \quad | \quad \tanh(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

Utilisées historiquement. Saturent très vite aux extrêmes, provoquant le phénomène d'évanouissement du gradient (*Vanishing Gradient*).

4. Dropout (Régularisation par Amnésie Sélective)

C'est **ici** qu'intervient l'extinction volontaire de certains neurones ! Lors de chaque passe d'entraînement, le Dropout désactive aléatoirement une proportion p (ex: 20% à 50%) des neurones de la couche.

- **Rôle** : Empêcher la co-adaptation complexe des neurones (apprentissage par cœur ou surapprentissage / *overfitting*).
- **Effet** : Force le réseau à répartir l'information représentative sur l'ensemble des neurones plutôt que de se reposer sur quelques neurones dominant la décision.

5. Batch Normalization (Normalisation par Lot)

Normalise les activations de chaque couche cachée pour avoir une moyenne de 0 et une variance de 1 sur le lot actuel :

$$\hat{x} = (x - \mu_B) / \sqrt{\sigma_B^2 + \epsilon} \rightarrow y = \gamma \hat{x} + \beta$$

Accélère considérablement la convergence et stabilise les réseaux très profonds.

7.5. Callbacks et Automatisation Dynamique

Les **Callbacks** sont des modules de surveillance exécutés à la fin de chaque lot ou époque pour contrôler dynamiquement le déroulement de l'apprentissage.

1. Early Stopping (L'Arrêt Prématuré)

Le garde-fou principal contre le surapprentissage.

- **Mécanisme** : Évalue la perte sur le jeu de validation (*val_loss*) à chaque époque. Si la *val_loss* cesse d'augmenter ou stagne pendant un nombre déterminé d'époques (appelé **patience**), l'entraînement est interrompu automatiquement.
- **Paramètres clés** :
 - *monitor* : La métrique à surveiller (ex: '*val_loss*' ou '*val_accuracy*').
 - *patience* : Nombre d'époques tolérées sans amélioration (ex: *patience* = 10).
 - *restore_best_weights* : Booléen forçant le réseau à charger à la fin les poids correspondant à la meilleure époque enregistrée plutôt qu'à la dernière.

2. Model Checkpoint (Sauvegarde Automatique)

Sauvegarde périodiquement l'état complet du réseau (structure + poids) sur le disque dur.

- **Paramètres clés** :
 - *filepath* : Chemin du fichier de destination (ex: *best_model.h5* ou *model.pt*).
 - *save_best_only* = True : Ne remplace le fichier sur le disque que si la métrique actuelle dépasse le record historique.

3. Learning Rate Schedulers (Planificateurs du Taux d'Apprentissage)

Ajustent le taux d'apprentissage α au fur et à mesure de l'avancement pour affiner la convergence.

- **ReduceLROnPlateau** : Réduit α par un facteur (ex: diviser par 2 ou 10) si la *val_loss* ne s'améliore pas pendant une certaine patience.

$$\alpha_{\text{nouveau}} = \alpha_{\text{actuel}} \cdot \text{factor}$$

- **Cosine Annealing** : Fait osciller ou décroître α selon une courbe cosinusoidale.
- **Warmup** : Augmente très progressivement α durant les premières époques avant d'entamer la décroissance, stabilisant le départ de grands réseaux (ex: Transformers).

8. Exemple de Code Complet et Structuré (PyTorch & TensorFlow)

Pour illustrer l'implémentation concrète de tous les concepts abordés (Logits, Softmax, Loss, Optimizer, Callbacks, Batching), voici deux fragments de code de référence prêts à l'emploi.

8.1. Implémentation en TensorFlow / Keras

```
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks

# 1. Définition de l'Architecture
model = models.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.3), # Amnésie volontaire (30% des neurones désactivés par batch)
    layers.BatchNormalization(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10)      # SORTIE : 10 Logits (Pas d'activation Softmax ici si intégrée à la
loss)
])

# 2. Configuration des Algorithmes (Loss & Optimizer)
# Loss: Categorical Cross-Entropy à partir des Logits bruts (from_logits=True pour stabilité
numérique)
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)

model.compile(optimizer=optimizer, loss=loss_fn, metrics=['accuracy'])

# 3. Configuration des Callbacks (Surveillance)
my_callbacks = [
    callbacks.EarlyStopping(
        monitor='val_loss',
        patience=8,
        restore_best_weights=True,
        verbose=1
    ),
    callbacks.ModelCheckpoint(
        filepath='best_model.h5',
        monitor='val_loss',
        save_best_only=True
    ),
    callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=4,
        verbose=1
    )
]

# 4. Lancement de l'Entraînement avec Batching
# history = model.fit(x_train, y_train, epochs=100, batch_size=32, validation_data=(x_val,
y_val), callbacks=my_callbacks)
```

8.2. Implémentation Explicite en PyTorch

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

# 1. Architecture PyTorch
class DeepClassifier(nn.Module):
    def __init__(self):
        super(DeepClassifier, self).__init__()
        self.fc1 = nn.Linear(784, 256)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.3)
        self.fc2 = nn.Linear(256, 10) # 10 Logits bruts

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        logits = self.fc2(x)
        return logits

model = DeepClassifier()

# 2. Loss & Optimizer
# PyTorch CrossEntropyLoss intègre directement la fonction Softmax + NLLLoss
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5, patience=4)

# 3. Boucle d'EntraînementManuelle avec Batch Size
# train_loader = DataLoader(dataset, batch_size=32, shuffle=True)

# Pour chaque Époque :
# for epoch in range(num_epochs):
#     model.train()
#     for batch_x, batch_y in train_loader:
#         optimizer.zero_grad()          # Réinitialisation des gradients
#         logits = model(batch_x)         # Forward pass -> Logits bruts
#         loss = criterion(logits, batch_y) # Loss Function (Calcul de la perte)
#         loss.backward()                 # Backpropagation (Calcul  $\partial L / \partial w$ )
#         optimizer.step()                # Optimizer (Mise à jour des poids w)
```

9. Synthèse des Bonnes Pratiques pour l'Entraînement

Pour garantir une convergence rapide, stable et éviter le surapprentissage lors du développement d'un modèle d'apprentissage profond, il est recommandé de suivre la feuille de route suivante :

1. **Choix de la fonction de perte** : Adapter la perte rigoureusement à la tâche (Cross-Entropy pour la classification, MSE/Huber pour la régression, Focal Loss pour les données fortement déséquilibrées).
2. **Choix de l'optimiseur** : Démarrer systématiquement avec **Adam** ou **AdamW** avec un taux d'apprentissage par défaut de 10^{-3} (**0.001**). Pour un polissage final très fin, basculer sur **SGD avec Momentum (0.9)**.
3. **Gestion de la sortie** : Calculer la perte directement sur les **Logits bruts** en activant les flags de stabilité numérique (ex: `from_logits=True` dans Keras) plutôt que d'appliquer une couche Softmax manuelle suivie d'une perte standard, afin de prévenir les erreurs d'arrondi numérique.
4. **Taille de lot (Batch Size)** : Sélectionner une taille de lot modérée (ex: 32 ou 64). Les lots plus petits introduisent un bruit bénéfique qui aide l'optimiseur à s'échapper des minima locaux peu profonds.
5. **Protection active** : Toujours intégrer un callback **Early Stopping** couplé à un **Model Checkpoint** pour capturer le meilleur modèle avant l'apparition du surapprentissage.